DATAMICS · MACHINE LEARNING · 8 SOLUTION NOTEBOOKS

How to read ML model results

Data and training · results on test data · where to look in the notebook

#	NOTEBOOK	MODEL	DATA (ROWS)
1	`3-Lineare_Regression-Projekt_Uebung-Loesung`	LinearRegression	Ecommerce Customers (500)
2	`3-Logistische_Regression_Projekt-Loesung`	LogisticRegression	advertising.csv (1,000)
3	`3-K_Nearest_Neighbors_Projekt-Loesung`	KNeighborsClassifier	KNN_Project_Data (~1,000)
4	`3-Decision_Trees_und_Random_Forests_Projekt-Loesung`	DecisionTree, RandomForest	loan_data.csv (9,578)
5	`3-Support_Vector_Machines_Projekt-Loesung`	SVC + GridSearchCV	Iris (150)
6	`3-K_Means_Clustering_Projekt-Loesung`	KMeans	College_Data (777)
7	`3-Nlp_Projekt-Loesung`	CountVectorizer/TF-IDF + MultinomialNB	Yelp (10,000)
8	`04-Keras-Projekt-Uebung-Loesung`	Keras Sequential (Dense)	LendingClub (396,030)

CHAPTER 0
How to read + cheat sheet
from page 3

CHAPTERS 1–10
8 notebooks + comparison
from page 5

✓ fine ! with caveats ✗ problem
calculated number calculated from the notebook's outputs · extra test own run on the same data

As of: 16.10.2026 · Basis: the outputs saved in the 8 notebooks (versions sent on 16.10.2026)

Contents

CHAPTERS

0	How to read this document + cheat sheet	3
1	Linear regression — e-commerce customers	5
2	Logistic regression — ad clicks	7
3	KNN — anonymised data	10
4	Decision tree and random forest — LendingClub	12
5	SVM — iris flowers	14
6	K-Means — US colleges	16
7	NLP — Yelp reviews	19
8	Keras — LendingClub (neural network)	21
9	Comparison of all models and the key lessons	25
10	Technical notes for re-running the notebooks	27

How to use it

Chapters 1–8 all have the same structure: key facts → "In short" (Data / Training / Test result) → 1) data and training → 2) results on the test data → what to watch out for → conclusion. References `In [n]` point to the cell in the notebook. Cheat sheet with the metrics: chapter 0.

ENGLISH VERSION

Interpretation of 8 notebooks

Data and training · results on test data · where to look

0 How to read this document + cheat sheet

For each of the 8 notebooks I answer two questions: **(1) were the data suitable for training and how did the training go**, and **(2) how to interpret the results on the test data**, i.e. on data the model did not see during training. Every conclusion refers to the place in the notebook where you can see it.

Structure of each chapter

- **Key facts** — file, task, model, problem type, data.
- **In short** — three ratings: Data / Training / Test result (✓ fine, ! with caveats, ✗ problem).
- **1) Data and training — where to look** — rows of "evidence": on the left the cell and the code, on the right *what you see → what it means*.
- **2) Results on the test data — how to read them** — confusion matrix, metrics, comparison with the baseline.
- **What to watch out for**, sometimes an **Extra test**, and finally the **Conclusion**.

References and labels

`In [5]` is the execution counter shown to the left of a cell in Jupyter. Next to it I give a code snippet so that you can find the cell with Ctrl+F. When you re-run the notebook, the numbers change — then search by the code. In the Keras notebook the numbers repeat (the file contains outputs from two runs), so the code snippet matters more there.

calculated — the number is not shown directly in the notebook; I calculated it from the visible outputs (e.g. the baseline from the `support` column). extra test — my own run of the code on the same, publicly available data (K-Means and NLP). It reproduced the notebook results exactly, so it is definitely the same data.

Cheat sheet 1 — are the data suitable for training?

QUESTION	WHERE TO LOOK	WARNING SIGN
How much data is there and is it complete?	`df.info()` : number of rows, *Non-Null Count*, *Dtype*	*Non-Null* smaller than the number of rows = missing values; `object` = text that has to be encoded
Are the values possible?	`df.describe()` : rows *min* and *max*	percentages > 100, negative times or amounts, *max* many times larger than the 75% quantile
Do the features have a similar scale?	`describe()` : *std*, *min*/*max*	differences of ×100–×1000 → scaling for KNN, K-Means, SVM, neural networks
Are the classes balanced?	`countplot` , `value_counts()` , the mean of a 0/1 column	one class ≥ 70–80% → accuracy misleads, compute the baseline
Do the features separate the classes?	`pairplot` , histograms and box plots with `hue=label` , `corr()` with the target	the class distributions almost overlap → weak model
Is there no data leakage?	the feature list; the order of `fit` and `train_test_split`	a feature known only "after the fact"; scaler / vectorizer / imputation fitted before the split

QUESTION	WHERE TO LOOK	WARNING SIGN
Will the evaluation be fair?	`train_test_split`: `test_size`, `random_state`, `stratify`	no `random_state` (results not reproducible); test set used for tuning

Cheat sheet 2 — how did the training go?

- In scikit-learn `fit()` prints nothing. Only the results tell you about quality. Quick test: compare `model.score(X_train, y_train)` with `model.score(X_test, y_test)`. **A large difference** = overfitting; **both scores low** = underfitting (the model is too simple or the data contain no signal).
- Learning curves (Keras): *loss* and *val_loss* fall together → good; *val_loss* stalls or rises while *loss* keeps falling → overfitting; both high → underfitting.
- Plot of the error against a hyperparameter (KNN): choose a value from a flat minimum, not from a single "dip".

Cheat sheet 3 — how to read results on the test data

	predicted 0	predicted 1
actual 0	**TN** correct 0	**FP** false alarm
actual 1	**FN** missed	**TP** correct 1

In scikit-learn **rows = truth, columns = prediction**. The classes are ordered by the sorted labels (0, 1 or 1, 5 or alphabetically). Diagonal = hits.

- **accuracy** = (TN + TP) / all
- **precision** (class 1) = TP / (TP + FP) — *when the model says "1", how often is it right?*
- **recall** (class 1) = TP / (TP + FN) — *what share of the real "1"s did it find?*
- **F1** = 2·P·R / (P + R) — a compromise between precision and recall
- **support** = number of real examples of the class in the test set
- **macro avg** = plain mean over the classes (every class equally important); **weighted avg** = mean weighted by support (dominated by the large class)
- **balanced accuracy** = mean of the recalls of all classes; with 2 classes 0.5 = guessing

- **Baseline** = share of the largest class in the test set (largest *support* / total). That is the accuracy of a "model" that always says the same thing. A model is worth something only clearly above this number.
- **Uncertainty** of accuracy: approx. $\pm 1.96 \cdot \sqrt{p \cdot (1-p)/n}$. For n = 45 that is ±6 pp, n = 300: ±4, n = 1,226: ±1.5, n = 7,905: ±0.7. Smaller differences are noise.
- **Regression**: MAE (mean error in the units of the target), MSE (squares — penalises large errors more), RMSE (square root of MSE, again in the units of the target), R^2 (share of explained variance: 1 = perfect, 0 = no better than the mean). Compare RMSE with the std of the target. The residuals (truth − prediction) should be symmetric around 0 and without a pattern.
- **Clustering**: cluster numbers are random. They are compared with labels via the best matching or with a measure that does not depend on the numbering (Adjusted Rand Index).

READING ORDER FOR A CLASSIFICATION RESULT

1. Confusion matrix: is every class predicted at all? (a column of zeros only = no)
2. Accuracy vs baseline.
3. Recall and precision of the most important class — usually the smaller one.
4. Macro avg / balanced accuracy.
5. Size of the test set (uncertainty).
6. Only then business conclusions.

1 Linear regression — e-commerce customers

File	3-Lineare_Regression-Projekt_Uebung-Loesung.ipynb
Task	predict a customer's yearly spending (`Yearly Amount Spent` , in $) from 4 behavioural features
Model	`LinearRegression` (scikit-learn)
Type	regression (the result is a number, not a class)
Data	500 customers; 4 numeric features + 3 text columns (not used)

IN SHORT

✓ **Data**	Very good: complete, clean, with clear linear relationships. The data were generated artificially — hence their "perfection".
✓ **Training**	Correct 70/30 split with `random_state=101` . A simple model (5 parameters) on 350 examples — overfitting practically ruled out.
✓ **Test result**	Very good: MAE ≈ $7.23, RMSE ≈ $8.93 with mean spending of $499; R^2 ≈ 0.99 calculated. The residuals look normal.

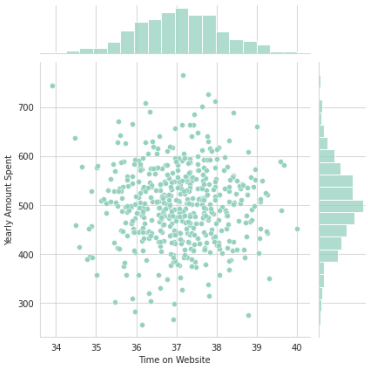
1) Data and training — where to look

`In [5]` `customers.info()`	**You see:** 500 rows, "500 non-null" in every column; 5 `float64` columns and 3 `object` (Email, Address, Avatar). → **Conclusion:** no missing data. The text columns are identifiers and random attributes — rightly they are not used in the model (In [14] selects only the 4 numeric features).
`In [4]` `customers.describe()`	**You see:** the four features have *std* ≈ 1.0 (0.99 / 0.99 / 1.01 / 1.00), and the mean ≈ the median (e.g. 33.05 vs 33.08). The ranges make sense (Length of Membership 0.27–6.92 years, no negative values). → **Conclusion:** symmetric distributions without absurd values. Such equal *std* values are a sign of generated data. An important consequence: all features have the same scale, so the coefficients can later be compared directly.
`In [7]` `jointplot(x='Time on Website', …)`	**You see:** a round cloud of points without a slope. → **Conclusion:** the time on the website is not related to spending. The code comment "*# Je mehr Zeit auf der Webseite, desto mehr Ausgaben*" contradicts the plot (and the later coefficient of 0.19). Rule: trust plots and numbers, not comments.
`In [8]` `jointplot(x='Time on App', …)`	**You see:** a cloud clearly sloping upwards. → **Conclusion:** more time in the app goes hand in hand with higher spending.
`In [10] / In [11]` `sns.pairplot(customers)`	**You see:** in the "Yearly Amount Spent" row the narrowest, diagonal cloud is the one with Length of Membership; pairs of input features form round clouds. → **Conclusion:** the strongest predictor is the length of membership (the notebook gives the same answer in In [11]). The features are not correlated with each other → no multicollinearity, the coefficients will be stable.
`In [12]` `lmplot(x='Length of Membership', …)`	**You see:** the points lie close to the line, with a very narrow confidence band. → **Conclusion:** the relationship is linear — linear regression is a good choice.
`In [16]` `train_test_split(X, y, test_size=0.3, random_state=101)`	**You see:** 350 training rows (confirmed by In [22] `print(X_train.head)` : "[350 rows x 4 columns]") and 150 test rows. → **Conclusion:** a sensible split; `random_state` makes the result reproducible.

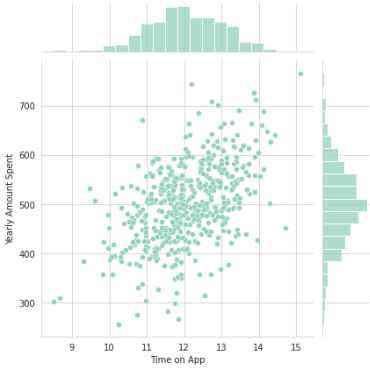
In [19]

```
lm.fit(X_train, y_train)
```

You see: only `LinearRegression()` — sklearn prints no training metrics. → **Conclusion:** we judge the training indirectly: 5 parameters for 350 examples = no risk of overfitting. Check: `lm.score(X_train, y_train)` and `lm.score(X_test, y_test)` should give almost the same R^2.



In [7] Time on Website vs. Yearly Amount Spent — a cloud without a trend: no relationship.



In [8] Time on App vs. Yearly Amount Spent — a clear rising trend.

2) Results on the test data — how to read them

In [22]

```
plt.scatter(y_test,
predictions)
```

You see: the points lie almost perfectly on the diagonal. → **How to read:** each point is a test customer: X axis — actual spending, Y axis — predicted spending. A perfect model = the line y = x. A narrow band = small errors. A curve would mean non-linearity, a "funnel" widening to the right — a growing error for large values. Nothing like that here.

In [23]

```
metrics.mean_absolute_err
or(…) /
mean_squared_error(…)
```

You see: MAE 7.23; MSE 79.81; RMSE 8.93. → **How to read:** MAE — the average error is $7.23; RMSE — the "typical" error is $8.93 (it penalises large errors more); MSE — the same in $², hard to interpret. Reference point: mean spending $499, std $79 (In [4]). A model that always predicts the mean would have RMSE ≈ $79 — ours is wrong about 9 times less, and the MAE is about 1.5% of mean spending. RMSE only slightly larger than MAE → no single huge errors.

MAE	RMSE	R^2 (estimate)	Test set
$7.23	**$8.93**	**≈ 0.99**	**150**
≈ 1.5% of the mean	vs ≈ $79 for "the mean"	calculated	customers

section "Das Modell bewerten"

You see: the text announces R^2 ("erklärte Varianz"), but the code does not compute it. → **Conclusion:** estimate calculated: $R^2 \approx 1 - MSE/Var(y) = 1 - 79.8 / 79.3^2 \approx 0.987$, i.e. the model explains about 99% of the variance in spending. Exactly: `metrics.r2_score(y_test, predictions)`.

In [30]

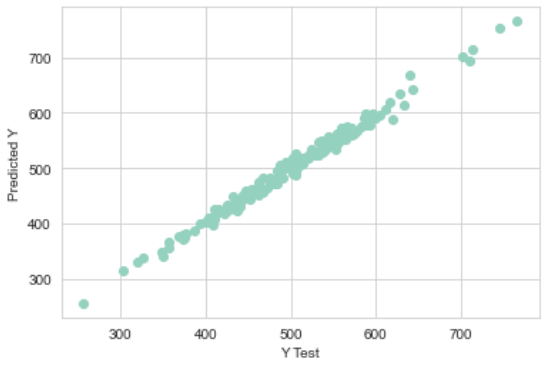
```
sns.histplot((y_test-
predictions), bins=50)
```

You see: a bell shape, centred at 0, range approx. −$27…+$30. → **How to read:** residual = truth − prediction. Centre at 0 → the model is not systematically wrong in either direction; symmetry and no long "tails" → the assumptions of linear regression are met. It is worth adding a plot of residual vs prediction (it should be a shapeless cloud).

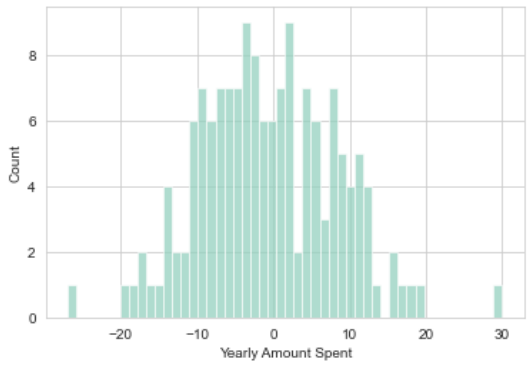
In [20] / In [25]

```
lm.coef_ /
pd.DataFrame(lm.coef_,
X.columns)
```

You see: Length of Membership 61.28; Time on App 38.59; Avg. Session Length 25.98; Time on Website 0.19. → **How to read:** with the other features unchanged: +1 year of membership ≈ +$61 per year, +1 min in the app ≈ +$39, +1 min of a session in the store ≈ +$26, +1 min on the website ≈ +$0.19 (practically zero). Comparing these numbers with each other is legitimate only because all features have *std* ≈ 1 (In [4]).



In [22] actual (X) vs predicted (Y) spending — the points lie on the diagonal.



In [30] histogram of the residuals — symmetric, centred at 0.

WHAT TO WATCH OUT FOR

- Such a good result on real data would be suspicious (e.g. target leakage into the features). Here it is due to the data having been generated from a linear formula with little noise.
- Correlation ≠ causation: the model does not say that "forcing" longer app use would raise spending.
- scikit-learn does not report the significance of the coefficients (p-values). The question "is 0.19 for the website different from zero?" is answered by statsmodels (OLS → `summary()`).

CONCLUSION

Good data → training without problems → a very good and reliable test result. For the business (section "Schlussfolgerungen"): spending is explained mainly by the length of membership and the time in the app; the website has no effect. Whether to develop the app or fix the website — the data alone cannot decide that (costs, causation). The notebook rightly points this out.

2 Logistic regression — ad clicks

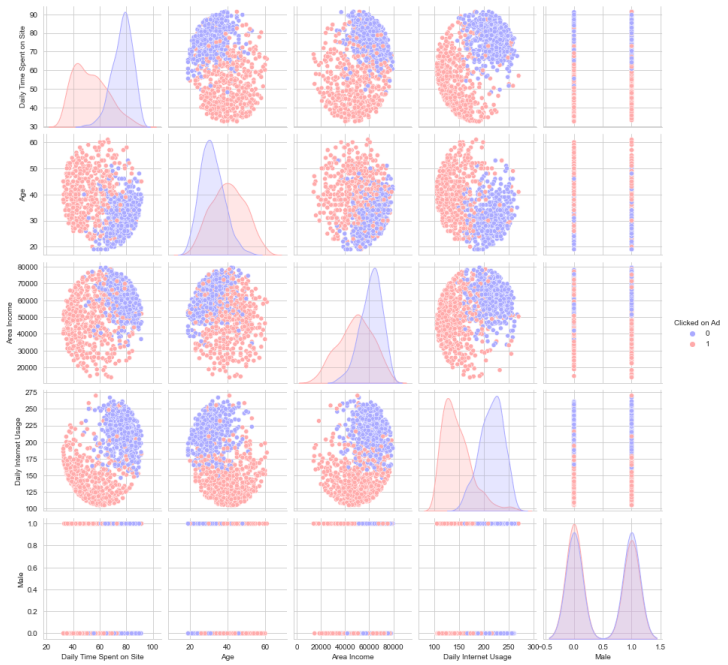
File	3-Logistische_Regression_Projekt-Loesung.ipynb
Task	predict whether a user clicks on an ad (`Clicked on Ad`: 1 = yes, 0 = no)
Model	`LogisticRegression`
Type	binary classification
Data	1000 users, 10 columns (fictitious data — "Fake-Daten")

IN SHORT

✓ **Data**	Very good: complete, the classes perfectly balanced (500/500), the features clearly separate the classes.
✓ **Training**	Correct 67/33 split (`random_state=42`). A small remark: no scaling of features with very different ranges.
✓ **Test result**	Accuracy 0.91 with a baseline of 0.51. Asymmetric errors: 25 missed clickers vs 6 false alarms (calculated).

1) Data and training — where to look

In [4] `ad_data.info()`	**You see:** 1000 rows, “1000 non-null” everywhere; 4 `object` columns (Ad Topic Line, City, Country, Timestamp). → **Conclusion:** no missing values. The text columns have hundreds of unique values — deliberately left out in In [17] (rightly for a simple model; from Timestamp one could extract e.g. the hour of the day).
In [5] `ad_data.describe()`	**You see:** Clicked on Ad: mean 0.50 → exactly 500 × 0 and 500 × 1. “Suspiciously round” means (65.0002; 55,000.00008; 180.0001). Age 19–61, Area Income approx. 14–79k. → **Conclusion:** balanced classes → accuracy will be a fair measure. The data are generated. Very different scales (income ~55,000, age ~36) — see “What to watch out for”.
In [7] `ad_data['Age'].hist(bins=30)`	**You see:** ages from 19 to 61, mostly 25–45. → **Conclusion:** a sensible distribution, no impossible values.
In [15] `sns.pairplot(ad_data, hue='Clicked on Ad')`	**You see:** (the most important plot) Daily Time Spent on Site and Daily Internet Usage form two clearly separated clouds; on the diagonal there are two shifted “humps”. The clickers are older (Age) and come from regions with lower income (Area Income). Male: identical distributions. → **Conclusion:** even before training you can see that the task is easy — an accuracy of about 90% can be expected. The Male feature carries no information.
In [13] `jointplot(x='Daily Time Spent on Site', y='Daily Internet Usage')`	**You see:** two clusters of points. → **Conclusion:** these are the same two groups (clickers / non-clickers) — good separability.
In [18] `train_test_split(X, y, test_size=0.33, random_state=42)`	**You see:** a test set of 330 examples (support 162 + 168 in In [23]). → **Conclusion:** the class proportion in the test set is still about 50/50; the result is reproducible.
In [20] `LogisticRegression().fit(X_train, y_train)`	**You see:** default settings, no scaling; no warnings in the output. → **Conclusion:** training completed. A linear model with 6 parameters on 670 examples — low risk of overfitting.



In [15] pairplot with hue='Clicked on Ad' — Daily Time Spent on Site and Daily Internet Usage separate the classes; Male (last column) does not.

2) Results on the test data — how to read them

`In [23]`

```
print(classification_report(y_test, predictions))
```

You see: class 0: precision 0.86, recall 0.96; class 1: precision 0.96, recall 0.85; accuracy 0.91; macro ≈ weighted ≈ 0.91; support 162 / 168. → **How to read:** class 1 (clicked): when the model says "will click", it is right in 96% of cases (precision), but it detects 85% of the actual clickers (recall) — it misses one in seven. Class 0: it recognises almost all non-clickers (96%). Macro ≈ weighted because the classes are equal.

baseline

You see: the largest class in the test set: 168/330 = 50.9% calculated. → **Conclusion:** the model reaches 90.6% — about 40 percentage points above "guessing".

Confusion matrix reconstructed from the report calculated

	predicted 0	predicted 1
actual: 0 (no click)	**156**	6
actual: 1 (clicked)	25	**143**

The notebook does not print it, but it follows unambiguously from precision, recall and support. Check: `confusion_matrix(y_test, predictions)`.

Accuracy
90.6%
299 of 330

Baseline
50.9%
calculated

Recall "click"
0.85
25 misses

Precision "click"
0.96
6 false alarms

matrix

You see: 6 false alarms (a click was predicted, but there was none) vs 25 misses (the user clicked, but the model says "no"). → **Conclusion:** the errors are asymmetric — the model is "cautious" in predicting a click.

business conclusions

You see: precision 0.96 and recall 0.85 for class 1. → **Conclusion:** if the ad should reach only those who will click (tight budget) — high precision is an advantage. If no potential customer may be missed — recall needs improving: you can lower the threshold (`predict_proba(X_test)[:,1] > 0.4` instead of the default 0.5). With 330 examples the uncertainty interval of the accuracy is about ±3 pp (0.87–0.94) calculated.

WHAT TO WATCH OUT FOR

- No scaling: Area Income (~55,000) and Age (~36) in one model. L2 regularisation (default `C=1`) then acts unevenly on the features, and the solver may converge more slowly. Better: `make_pipeline(StandardScaler(), LogisticRegression())`.
- The notebook shows neither the confusion matrix nor the coefficients (`logmodel.coef_`) — yet they tell you where the model makes mistakes and which features "push" towards a click.

CONCLUSION

Good, well-separated data → correct training → a very good and reliable test result (balanced classes, so accuracy does not deceive). The only weakness: one in seven clickers is missed.

3 KNN — anonymised data

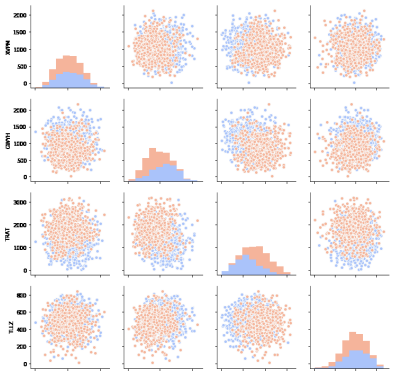
File	3-K_Nearest_Neighbors_Projekt-Loesung.ipynb
Task	predict `TARGET CLASS` (0/1) from 10 anonymised features
Model	`KNeighborsClassifier`; choice of K with the "elbow" method
Type	binary classification
Data	approx. 1000 rows (inferred from 300 = 30% test), artificial data

IN SHORT

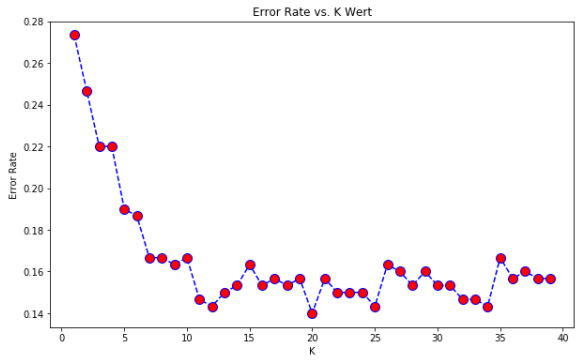
! **Data**	Probably correct, but not checked (no `info()` / `describe()`). The classes are balanced, but they overlap strongly in every pair of features.
! **Training**	Scaling done (necessary for KNN), but: the scaler was fitted on all the data, the split has no `random_state`, K was chosen on the test set.
✓ **Test result**	K=1: 72.7%, K=24: 85.0% with a baseline of 51.3%. A good result, but slightly optimistic.

1) Data and training — where to look

`In [3]` `df.head()`	**You see:** columns XVPM, GWYH, TRAT... (artificial names) and very different ranges (e.g. TLLZ approx. 280–725, TRAT approx. 610–2640 in the first rows). → **Conclusion:** the features have different scales, and KNN computes distances — scaling is mandatory.
missing `df.info() / df.describe()`	**You see:** the notebook goes straight to the plot. → **Conclusion:** from the notebook we do not know whether there are missing values or outliers. That is a gap in the EDA — always check it.
`In [4]` `sns.pairplot(df, hue='TARGET CLASS')`	**You see:** in every 2D plot the clouds of the two classes largely overlap; the distributions on the diagonal are only slightly shifted (most of all e.g. TRAT, GWYH). → **Conclusion:** no single feature and no pair of features separates the classes — the information is "spread" over many dimensions. Expect a medium result and a strong influence of the choice of K. The heights of the bars on the diagonal do not tell you the class sizes (the histograms are drawn together); the balance is confirmed only by the support 154/146 in In [30].
`In [7] → In [9]` `scaler.fit(df.drop('TARGET CLASS', axis=1))`	**You see:** after scaling (In [9] `df_feat.head()`) the values lie around 0 (approx. −1.8...2.2). → **Conclusion:** the scales are equalised — good. But there is a methodological error: the scaler was fitted on ALL the data before the split (the mean and std "saw" the test set). With 1000 rows the effect is minimal; correctly: first the split, then `scaler.fit(X_train)` and `transform` on both sets (as the Keras notebook does).
`In [23]` `train_test_split(scaled_features, …, test_size=0.30)`	**You see:** no `random_state`. → **Conclusion:** every run = a different split and different numbers. When you re-run the notebook you will see different results than in the notebook — that is normal, not an error.
`In [25] / In [26]` `KNeighborsClassifier(n_neighbors=1).fit(…)`	**You see:** KNN does not learn parameters — it memorises the training points. → **Conclusion:** with K=1 the accuracy on the training set would be 100% (each point is its own nearest neighbour). Textbook overfitting — only the test shows the truth.



In [4] part of the pairplot (features XVPM, GWYH, TRAT, TLLZ) — the class clouds overlap.



In [32] test error vs K — a steep drop down to K≈11, then a flat "plateau".

2) Results on the test data — how to read them

In [29] — K = 1

	pred. 0	pred. 1
actual: 0	**113**	41
actual: 1	41	**105**

In [33] — K = 24

	pred. 0	pred. 1
actual: 0	**131**	23
actual: 1	22	**124**

`In [29] / In [30]` `K = 1`	**You see:** 41 + 41 errors out of 300 → accuracy 72.7% (calculated); precision and recall of both classes approx. 0.72–0.73 (symmetric errors). → **Conclusion:** 100% on the training set, 73% on the test set → overfitting: the model reacts to the noise of individual neighbours.
`In [31] / In [32]` `plot "Error Rate vs. K Wert"`	**You see:** the error falls from approx. 0.27 (K=1) to approx. 0.15 for K ≥ 11, then a "plateau" of 0.14–0.165 with jagged jumps. → **How to read:** choose K from the flat area, not from a single "dip". A difference of 0.14 vs 0.16 is about 6 examples out of 300 — noise. K=20 (minimum approx. 0.14) is not really better than K=24 (0.15).
`In [31]` `pred_i = knn.predict(X_test)`	**You see:** the loop computes the error on `X_test` / `y_test`, i.e. the test set is used to choose K. → **Conclusion:** the test set is no longer "untouched" — the final 85% is slightly inflated. Correctly: `GridSearchCV(KNeighborsClassifier(), {'n_neighbors': range(1, 40)}, cv=5)` on the training set, and the test set only once, at the end.
`In [33]` `K = 24`	**You see:** 255/300 = 85.0% (calculated); precision and recall of both classes approx. 0.85; the errors are almost symmetric (23 vs 22). → **Conclusion:** the model does not favour either class. Baseline 154/300 = 51.3% (calculated) → model +34 pp; uncertainty approx. ±4 pp.
`In [30] / In [33]` `row "avg / total"`	**You see:** a different report layout than in the other notebooks. → **Conclusion:** this is the format of an older scikit-learn version; today you will see *accuracy / macro avg / weighted avg*.

K = 1	K = 24	Baseline	Uncertainty
72.7%	**85.0%**	**51.3%**	**±4 pp**
overfitting	255 of 300	calculated	calculated, n = 300

WHAT TO WATCH OUT FOR

- Three small methodological errors at once: the scaler before the split, no `random_state`, the choice of K on the test set. Each on its own is small; together they give a slightly optimistic and non-reproducible result.
- When you re-run the notebook, expect different numbers (for K from the plateau around 82–88%) — an effect of the random split.

CONCLUSION

More difficult data (the classes overlap), but after scaling and choosing K the model works decently: 85% against a 51% baseline. K=1 is a textbook example of overfitting, and the plot of the error against K is the textbook way to detect it.

4 Decision tree and random forest — LendingClub

File	3-Decision_Trees_und_Random_Forests_Projekt-Loesung.ipynb
Task	predict whether a loan will NOT be fully repaid (`not.fully.paid` = 1)
Models	`DecisionTreeClassifier()` (no limits) and `RandomForestClassifier(n_estimators=600)`
Type	binary classification, imbalanced classes
Data	9,578 LendingClub loans 2007–2010; 13 features + target

IN SHORT

! **Data**	Technically clean (zero missing values), but difficult: only 16% of the loans are unpaid, and the features distinguish them poorly (EDA).
! **Training**	A tree without limits → overfitting. The random forest in practice learned to say "will repay".
✗ **Test result**	Both models are useless for detecting risk: the tree 73% (below the baseline of 84.6%), the forest 84.55% = baseline, and it detects 2% of the bad loans.

1) Data and training — where to look

In [3] `loans.info()`	**You see:** 9,578 rows, 14 columns, "9578 non-null" everywhere; one `object` column: purpose. → **Conclusion:** no missing values (the data were cleaned beforehand — the introduction says so). purpose has to be converted into numbers: In [13] `pd.get_dummies(…, drop_first=True)`, In [14] shows 19 columns.
In [4] `loans.describe() – column not.fully.paid`	**You see:** mean = 0.160. → **Conclusion:** the most important number in the notebook: only 16% of the loans are class 1. The "always 0" model already reaches about 84% accuracy (= baseline). Accuracy will be misleading here — you have to look at the recall and precision of class 1.
In [4] `loans.describe() – min/max`	**You see:** revol.bal max 1,207,359 with a mean of 16,914; inq.last.6mths max 33; delinq.2yrs max 13; revol.util max 119. → **Conclusion:** strongly skewed distributions with outliers (revol.util > 100% is possible when the limit is exceeded, but it is worth checking). For trees this is a small problem — they split by thresholds, so the scale and outliers change little; scaling is unnecessary.
In [6] `hist FICO by credit.policy`	**You see:** everyone with credit.policy = 1 has a FICO score of about 660 or higher; below that there is only credit.policy = 0. → **Conclusion:** credit.policy is largely a threshold rule on FICO — the two features carry partly the same information.
In [7] `hist FICO by not.fully.paid`	**You see:** the distributions of the two groups almost overlap; the unpaid loans are only slightly shifted towards lower FICO. → **Conclusion:** FICO alone distinguishes the classes poorly.
In [8] `countplot(x='purpose', hue='not.fully.paid')`	**You see:** the blue bars (unpaid) are small in every category; small_business has a clearly higher share (approx. ¼), credit_card and major_purchase the lowest (approx. 1/10). → **Conclusion:** the purpose of the loan carries some information, but only weak.

In [9] / In [10]

```
lmplot(y='int.rate',
x='fico',
col='not.fully.paid')
```

You see: a strong downward trend (higher FICO → lower interest rate), but the panels for not.fully.paid = 0 and = 1 look similar. → **Conclusion:** no clear separation. EDA summary: a weak signal and a small target class → expect weak results.

In [16]

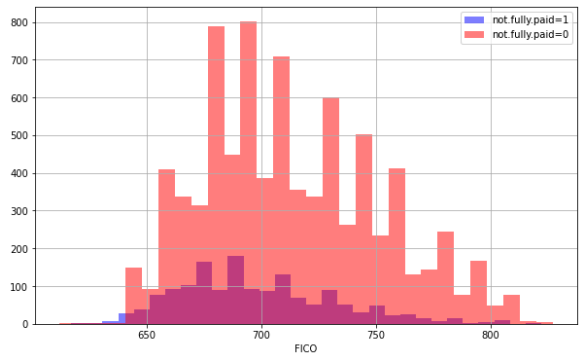
```
train_test_split(X, y,
test_size=0.30,
random_state=101)
```

You see: test set: 2,874 loans (support 2,431 + 443). → **Conclusion:** 15.4% class 1 in the test set — similar to the whole dataset (without `stratify`, but the proportions match anyway).

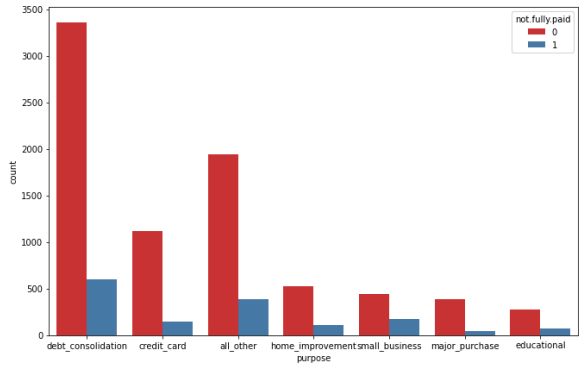
In [18] / In [25]

```
DecisionTreeClassifier()
/
RandomForestClassifier(n_
estimators=600)
```

You see: default settings: a tree without a depth limit, no `class_weight`, no `random_state`. → **Conclusion:** the tree grows until almost every leaf is "pure" → it memorises the training data (accuracy ≈ 100% on the training set; check `dtree.score(X_train, y_train)`). Without `random_state` the results differ slightly with every run.



In [7] FICO by not.fully.paid — the distributions almost overlap.



In [8] loan purpose by not.fully.paid — class 1 (blue) is small everywhere.

2) Results on the test data — how to read them

In [23] — decision tree

	pred. 0	pred. 1
actual: 0 (repaid)	**1,993**	438
actual: 1 (not repaid)	338	**105**

In [30] — random forest

	pred. 0	pred. 1
actual: 0 (repaid)	**2,421**	10
actual: 1 (not repaid)	434	**9**

In [22] / In [23]

```
decision tree
```

You see: of 443 unpaid loans the tree detected 105 (recall 0.24); of its 543 "will not repay" alarms, 105 were correct (precision 0.19). → **How to read:** random flagging would give a precision of approx. 0.15 (the share of class 1) — the tree is only about 1.25 times better than chance. The accuracy of 0.73 is LOWER than the baseline of 0.846. Macro avg F1 0.53 ≈ chance level.

In [29] / In [30]

```
random forest
```

You see: an accuracy of 0.85 looks good, but (2,421 + 9) / 2,874 = 84.55%, while "always 0" gives 2,431 / 2,874 = 84.59% calculated. → **How to read:** the forest is ONE example worse than a model that always says "will repay". It predicted class 1 only 19 times (9 correctly): recall 0.02 → it detects 2% of the bad loans. Balanced accuracy 0.51 calculated (0.50 = guessing).

Baseline	Tree	Random forest	Balanced acc.
84.6%	**73.0%**	**84.55%**	**0.53 / 0.51**
calculated: "always 0"	recall(1) 0.24	recall(1) 0.02	calculated

why "0.85" misleads

You see: with 85% in one class, accuracy mainly measures how well the majority class is recognised. → **Conclusion:** always look at the class 1 row in the report and at macro avg (F1 of the forest: 0.48).

In [31]

```
# Kommt darauf an welche
Metrik wir optimieren
```

You see: the notebook answers that it depends on the metric and that neither model was good. → **Conclusion:** by accuracy — the forest wins (but = baseline); by the recall of class 1, i.e. detecting the risk that matters to an investor — the tree wins (0.24 vs 0.02). Neither is fit for use.

question before In [28]

```
„Erhälst du etwas
komisches oder eine Art
Warnung?“
```

You see: the forest's report has no warnings. → **Conclusion:** what is "strange" is precisely that the forest almost never predicts class 1. If it had not predicted it a single time, sklearn would print an *UndefinedMetricWarning* (precision = 0/0).

WHAT TO WATCH OUT FOR / HOW TO IMPROVE

- `class_weight='balanced'` (tree and forest) — an error on class 1 costs more.
- Limit the tree: `max_depth`, `min_samples_leaf` — less overfitting.
- Instead of `predict()`: `predict_proba()` and your own threshold (e.g. 0.2–0.3) chosen on the precision-recall curve; evaluation with ROC-AUC / average precision.
- Cross-validation (`StratifiedKFold`) instead of a single split.

CONCLUSION

Clean but not very informative, imbalanced data → the tree overfits, the forest degenerates into "always repays". The forest's high accuracy is an illusion — a textbook example of why a result must always be compared with the baseline.

5 SVM — iris flowers

File	3-Support_Vector_Machines_Projekt-Loesung.ipynb
Task	recognise the iris species (setosa / versicolor / virginica) from 4 flower measurements
Model	`SVC()` (RBF kernel) + `GridSearchCV` for C and gamma
Type	multi-class classification (3 classes)
Data	the Iris dataset: 150 flowers, 50 of each species

IN SHORT

✓ **Data**	Exemplary: complete, balanced; setosa perfectly separated, versicolor and virginica overlap slightly.
✓ **Training**	Correct; GridSearch with 5-fold cross-validation. Printing `best_params_` is missing.
✓ **Test result**	43/45 (95.6%) with the default SVC, 42/45 (93.3%) after GridSearch — a difference of one flower is noise, not a deterioration.

1) Data and training — where to look

In [4]

```
sns.load_dataset('iris')
```

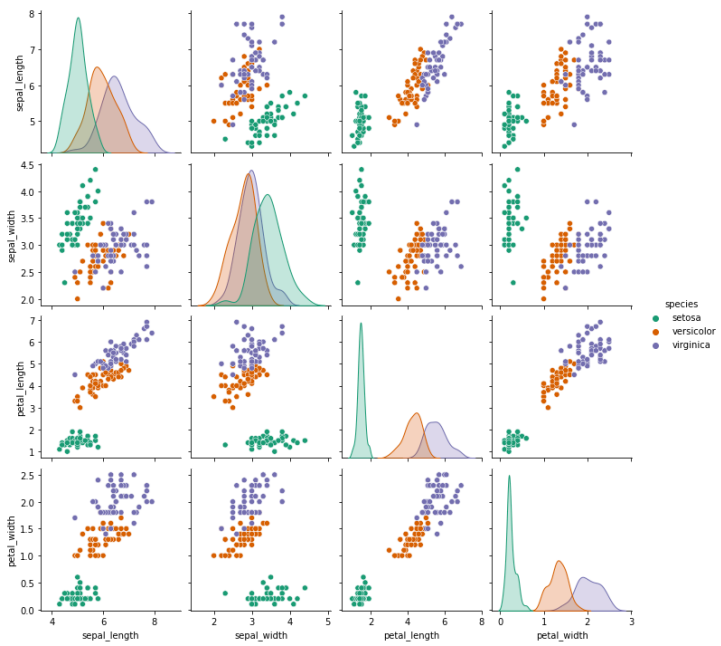
You see: the notebook calls neither `info()` nor `describe()`. → **Conclusion:** Iris is known to be clean (150 × 5, no missing values), but it is good practice to check it anyway — with other data this would be a mistake.

In [6]

```
sns.pairplot(iris,
hue='species')
```

You see: setosa (green) is completely separated in petal_length and petal_width (petals shorter than 2 cm and narrower than approx. 0.6 cm); versicolor and virginica overlap slightly (petal_length approx. 4.5–5 cm). → **Conclusion:** setosa will be classified without errors; the only possible mistakes are versicolor ↔ virginica. This is how the EDA predicts the structure of the confusion matrix.

In [7] `sns.kdeplot(setosa[…], …)`	**You see:** the distribution of sepal_width vs sepal_length for setosa only. → **Conclusion:** an illustration, no influence on the model.
feature scales	**You see:** everything in cm, from 0.1 to 7.9. → **Conclusion:** similar ranges, so the lack of scaling does no harm here (for SVM it is generally recommended: `StandardScaler` in a Pipeline).
In [9] `train_test_split(X, y, test_size=0.30)`	**You see:** no `random_state` and no `stratify`; the test set has 11 setosa, 17 versicolor and 17 virginica (support in In [16]). → **Conclusion:** 45 flowers is a very small test set: 1 flower = 2.2 pp of accuracy. Every run gives a different split and slightly different results.
In [19] `GridSearchCV(SVC(), param_grid, refit=True, verbose=2)`	**You see:** "Fitting 5 folds for each of 16 candidates, totalling 80 fits"; `grid.best_params_` and `grid.best_score_` are not printed. → **Conclusion:** we do not know which C and gamma were chosen or what the cross-validation score was — add these two lines. With 105 training examples each validation fold has about 21 flowers, so the CV scores "jump" in steps of about 5 pp and many combinations tie.



In [6] pairplot with `hue='species'` — setosa clearly apart; versicolor and virginica touch in the petal dimensions.

2) Results on the test data — how to read them

In [15] — default SVC

actual ↓ / pred. →	setosa	versic.	virgin.
setosa	**11**	0	0
versicolor	0	**15**	2
virginica	0	0	**17**

In [21] — after GridSearch

actual ↓ / pred. →	setosa	versic.	virgin.
setosa	**11**	0	0
versicolor	0	**14**	3
virginica	0	0	**17**

In [15] / In [16] `default SVC`	**You see:** diagonal 11 + 15 + 17 = 43 of 45 (95.6%); the only error: 2 versicolor classified as virginica. → **How to read:** in the report this is the same error seen from two sides: versicolor recall 0.88 (15/17) and virginica precision 0.89 (17/19).
In [21] / In [22] `after GridSearch`	**You see:** 3 errors (all versicolor → virginica), 42/45 = 93.3%. → **Conclusion:** to the notebook's question "Konnten wir uns verbessern?": no — but it is not a real deterioration either. The difference is one flower.

uncertainty

You see: with n = 45 the 95% interval for accuracy is approx. ±6–7 pp calculated. → **Conclusion:** the results of 95.6% and 93.3% are statistically indistinguishable. The notebook rightly says that this is primarily a GridSearch exercise.

how to compare

```
cross_val_score(model, X, y, cv=10)
```

You see: a single train/test split on 150 samples. → **Conclusion:** with small data, compare models with cross-validation (mean ± std over 10 splits), not with a single test.

SVC	GridSearch	Baseline	Uncertainty
95.6%	**93.3%**	**37.8%**	**±6–7 pp**
43 of 45	42 of 45	calculated: 17 of 45	calculated, n = 45

WHAT TO WATCH OUT FOR

- A small test set and no `random_state` → the result can change by a few points with every run.
- GridSearch chooses the parameters based on cross-validation, not on the test — a worse result on 45 examples does not mean that the choice was bad.

CONCLUSION

Ideal, well-separated data → training without problems → a very good test result (93–96%). The only errors appear exactly where the pairplot showed the classes overlapping (versicolor/virginica).

6 K-Means — US colleges

File	3-K_Means_Clustering_Projekt-Loesung.ipynb
Task	without using the labels, split 777 colleges into 2 clusters and check whether they match the private/public split
Model	`KMeans(n_clusters=2)`
Type	unsupervised learning (clustering); the `Private` label is used only for evaluation
Data	College_Data: 777 colleges, 17 numeric features

IN SHORT

! **Data**	Complete, but with erroneous values (Grad.Rate 118%, PhD 103%) and with feature scales that differ by up to a factor of a thousand.
! **Training**	K-Means on 17 unscaled features → the clusters are determined by the “college size” features, not by the private/public status.
✗ **Result**	The notebook shows an accuracy of 0.22, but that is an artefact of the cluster numbering. After correct matching: 0.78 with a baseline of 0.73 → the clusters correspond only weakly to the private/public split.

1) Data and training — where to look

`In [26]`

```
df.info()
```

You see: 777 rows (the introduction says 770 — a small inconsistency), non-null everywhere; Private as `object` (Yes/No). → **Conclusion:** no missing values.

`In [27]`

`df.describe() — row max`

You see: Grad.Rate max = 118 (a graduation rate > 100% is impossible) and PhD max = 103 (a share of faculty with a PhD > 100%). → **Conclusion:** two errors in the data. The notebook finds the first one (In [32]: Cazenovia College) and corrects it to 100. It does not notice the second — that is Texas A&M University at Galveston (I checked). Lesson: look at the min/max of EVERY column, especially the percentage ones.

`In [27]`

`df.describe() — row std`

You see: Apps ≈ 3,870, F.Undergrad ≈ 4,850, Expend ≈ 5,222, Outstate ≈ 4,023, but S.F.Ratio ≈ 4, Grad.Rate ≈ 17. → **Conclusion:** K-Means computes Euclidean distances, so the features with large numbers will dominate the result. The notebook does not scale the data.

`In [29] / In [30]`

```
lmplot('Outstate',
'F.Undergrad',
hue='Private'); hist
Outstate
```

You see: a clear split: public colleges are large (F.Undergrad up to 30k) and cheap for out-of-state students (Outstate usually < 10k); private ones are small and more expensive. In [28] (Room.Board vs Grad.Rate): a large overlap. → **Conclusion:** the EDA shows which features carry the private/public split: Outstate and F.Undergrad.

`In [33] / In [34]`

```
df['Grad.Rate']
['Cazenovia College'] =
100
```

You see: a *SettingWithCopyWarning*; In [34] confirms the correction (empty table). → **Conclusion:** in this older pandas version it worked. In pandas ≥ 3.0 such a "chained" assignment does NOT change the DataFrame (I checked) — use `df.loc['Cazenovia College', 'Grad.Rate'] = 100`.

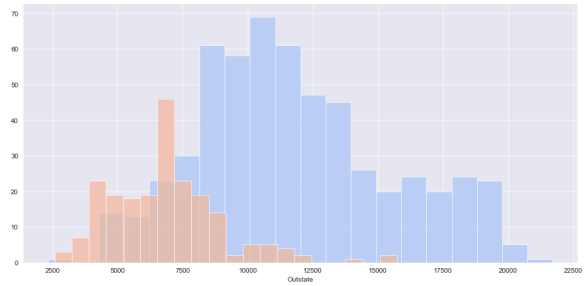
`In [37] / In [38]`

```
KMeans(n_clusters=2).fit(
df.drop('Private',
axis=1))
```

You see: no scaling and no `random_state`. → **Conclusion:** (a) the clusters will be dominated by the features with large numbers; (b) the numbering of the clusters (which one is "0" and which "1") is random and can change with every run.



In [29] Outstate vs F.Undergrad — public colleges (orange) large and cheaper, private ones (blue) small and more expensive.



In [30] histogram of Outstate by Private — the best single separating feature.

2) Results — how to read them

`In [39]`

`kmeans.cluster_centers_`

You see: two rows of 17 numbers (scientific notation). Cluster 1: Apps ≈ 10,363, Accept ≈ 6,551, Enroll ≈ 2,570, F.Undergrad ≈ 13,062; cluster 0: Apps ≈ 1,813, F.Undergrad ≈ 2,189. Outstate: 10,396 vs 10,719 — almost identical! → **How to read:** compare the columns between the rows. K-Means split the colleges into "large" and "small", not into "expensive private" and "cheap public". calculated: 97% of the difference between the centres (in the Euclidean metric) comes from F.Undergrad, Apps, Accept and Expend; the share of Outstate ≈ 0%.

In [43] — as in the notebook

	cluster 0	cluster 1
actual: public (0)	**138**	74
actual: private (1)	531	**34**

The same clusters, labels swapped calculated

	cluster 1 → "0"	cluster 0 → "1"
actual: public (0)	**74**	138
actual: private (1)	34	**531**

In [43] `confusion_matrix(df['Cluster'], kmeans.labels_)`	**You see:** cluster 0 (669 colleges) = 531 private + 138 public; cluster 1 (108 colleges) = 74 public + 34 private. → **Conclusion:** cluster 1 is simply the large colleges (mostly public).

In [44]
`classification_report(…)`
`→ accuracy 0.22`
	You see: the notebook assumes "cluster 1 = private" and gets an accuracy of 0.22. → **How to read:** K-Means knows nothing about labels — it assigns the numbers 0/1 randomly. With the reverse assignment (cluster 0 = private, cluster 1 = public): (531 + 74) / 777 = 77.9% calculated. extra test: on the same data, with 20 different `random_state` values, exactly this matrix came out 13 times and the swapped one 7 times. So the same model can show "22%" or "78%".

proper evaluation	**You see:** the best matching gives 77.9% vs the baseline "always private" 565/777 = 72.7% calculated. → **Conclusion:** only +5 pp above the trivial answer. Public colleges are recognised in 35% of cases (74 of 212). Adjusted Rand Index (a measure independent of the numbering: 0 = random, 1 = perfect) = 0.23 → weak agreement.
In [44] „Gar nicht so schlecht…"	**You see:** the notebook's comment praises the result. → **Conclusion:** 22% is not "not bad at all" — it is swapped labels; after swapping them, the result is barely better than the baseline. This comment is misleading.

Notebook (In [44])	After matching	Baseline	ARI
22.1%	**77.9%**	**72.7%**	**0.23**
labels swapped	calculated	"always private"	calculated

EXTRA TEST (OUTSIDE THE NOTEBOOK, SAME DATA)

- **All 17 features after `StandardScaler`:** K-Means splits the colleges into "elite/selective" and the rest (largest differences between the centres: Top10perc, Top25perc, PhD, Terminal, Outstate). The agreement with the private/public split is even worse: 52%, ARI ≈ 0.
- **Only the 2 features identified by the EDA** (Outstate + F.Undergrad, scaled): 87.4% agreement, ARI 0.52.
- **Lesson:** K-Means finds the dominant structure in the features it is given, not "the one we are interested in". The choice of features and the scaling decide the result.

WHAT TO WATCH OUT FOR

- In a real application (without labels) there is no confusion matrix — the notebook itself notes that this is only a "theoretical" check. The quality of the clusters is then assessed e.g. with the *silhouette score*, the elbow method (inertia vs k) and an interpretation of the centres.
- When comparing clusters with labels, use measures that do not depend on the numbering (`adjusted_rand_score`) or the best matching of the labels.

CONCLUSION

The data needed cleaning (2 impossible values) as well as scaling and a choice of features. The clusters reflect the size of the colleges, not their status. The result of 0.22 is an artefact of the numbering; realistically it is 0.78 with a baseline of 0.73 — weak.

7 NLP — Yelp reviews

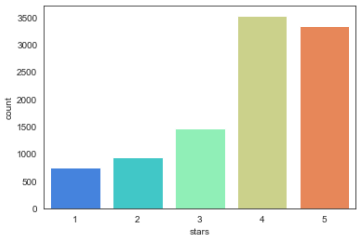
File	3-Nlp_Projekt-Loesung.ipynb
Task	recognise 1 star vs 5 stars from the text of a review
Models	(1) `CountVectorizer` + `MultinomialNB`; (2) Pipeline: `CountVectorizer` + `TfidfTransformer` + `MultinomialNB`
Type	text classification (binary), imbalanced classes
Data	10,000 Yelp reviews; 4,086 for the task (749 × 1★, 3,337 × 5★)

IN SHORT

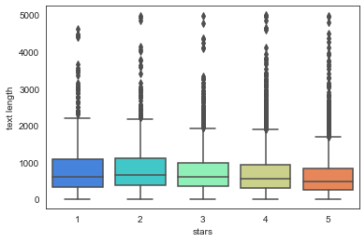
! **Data**	Complete (zero missing values), but imbalanced: only 18% of the reviews have 1★.
! **Training**	Model 1 correct (with a small leak: the vocabulary is built on all the data). Model 2 (TF-IDF) degenerate.
! **Test result**	Model 1: 92.6% with a baseline of 81.4%, but it misses 30% of the 1★ reviews. Model 2: always "5★" = exactly the baseline — useless, and the report in the notebook hides it.

1) Data and training — where to look

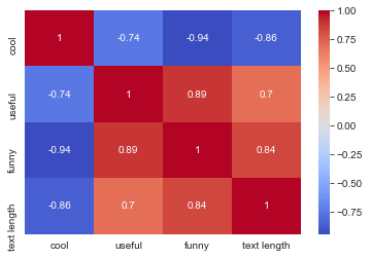
`In [4]` `yelp.info()`	**You see:** 10,000 rows, non-null everywhere; text as `object`. → **Conclusion:** no missing values.
`In [5]` `yelp.describe()`	**You see:** stars average 3.78; cool/useful/funny — max 77/76/57 with a median of 0–1. → **Conclusion:** the ratings are skewed towards positive; the votes are strongly skewed (they are not used in the model anyway).
`In [8] / In [9]` `FacetGrid(col='stars') / boxplot text length`	**You see:** similar histogram shapes, almost equal median text lengths for 1–5★, many outliers. → **Conclusion:** the length of the text distinguishes the ratings poorly — it would not be a good feature. (The different bar heights in the FacetGrid are simply the different class sizes.)
`In [10]` `sns.countplot(x='stars')`	**You see:** 4★ and 5★ dominate (approx. 3,500 and 3,300), 1★ is the smallest group (approx. 750). → **Conclusion:** in the 1★ vs 5★ task the classes are imbalanced: 749 vs 3,337, i.e. 18.3% are 1★ (calculated). Baseline "always 5★" ≈ 81%.
`In [11] → In [13]` `groupby('stars').mean() → corr() → heatmap`	**You see:** correlations of the order of −0.94 and 0.89. → **Conclusion:** careful: they are computed on 5 rows (the means for 1…5 stars), not on 10,000 reviews. With n = 5 such high correlations can occur by chance; it is a correlation of group means, not a relationship between individual reviews (ecological fallacy). Do not draw conclusions about the model from this heatmap.
`In [14]` `yelp[(yelp.stars==1) | (yelp.stars==5)]`	**You see:** only 1★ and 5★ remain. → **Conclusion:** 4,086 reviews to classify.
`In [17] → In [19]` `X = cv.fit_transform(X), then train_test_split`	**You see:** the vocabulary (extra test: 19,183 words) was built from the test reviews too. → **Conclusion:** a small leak. Correctly: first the split, then fit on the training set only (as the Pipeline does in the second part). extra test: with the correct order the result is almost identical ([[155, 73], [15, 983]], 92.8%) — here the leak changed nothing, but the habit matters.
`In [19]` `train_test_split(X, y, test_size=0.3, random_state=101)`	**You see:** test = 1,226 reviews (228 × 1★, 998 × 5★ — support in In [24]). → **Conclusion:** the proportions are the same as in the whole dataset; the result is reproducible.



In [10] number of reviews by stars — 1★ is the smallest group.



In [9] text length by stars — the medians are almost equal.



In [13] correlation heatmap — computed on only 5 means.

2) Results on the test data — how to read them

In [24] — model 1: Count + NB

	pred. 1★	pred. 5★
actual: 1★	**159**	69
actual: 5★	22	**976**

In [31] — model 2: TF-IDF + NB

	pred. 1★	pred. 5★
actual: 1★	**0**	228
actual: 5★	**0**	**998**

A column of zeros only: the model never chooses 1★.

`In [24]`

`model 1 — confusion matrix`

You see: diagonal 159 + 976 = 1,135 hits out of 1,226 = 92.6%. 22 5★ reviews classified as 1★; 69 1★ reviews classified as 5★. → **How to read:** rows = truth, columns = prediction, class order 1★, 5★.

`In [24]`

`model 1 — classification_report`

You see: 1★: precision 0.88, recall 0.70; 5★: precision 0.93, recall 0.98; accuracy 0.93; macro avg F1 0.87, weighted 0.92. → **How to read:** when the model says "1★", it is right in 88% of cases (159 of 181). But of 228 real negative reviews it detects 159 — it misses one in three. If the goal is to catch dissatisfied customers, the recall of 1★ is exactly what matters. The difference between macro (0.87) and weighted (0.92) reveals the weaker small class.

baseline

You see: 998 / 1,226 = 81.4% calculated. → **Conclusion:** model 1 is 11 pp better — it really learned something. Balanced accuracy 0.84 calculated.

Model 1	Baseline	Recall 1★	Model 2
92.6%	**81.4%**	**0.70**	**81.4%**
accuracy	calculated	69 misses	= baseline, recall 1★ = 0

`In [31]`

`model 2 — matrix [[0, 228], [0, 998]]`

You see: the first column is all zeros. → **How to read:** the model NEVER predicts 1★. It is a constant classifier: accuracy = 998 / 1,226 = 81.4% = exactly the baseline; balanced accuracy 0.50 calculated = guessing level.

`In [31]`

`classification_report(…, labels=np.unique(predictions))`

You see: only the class 5 row (precision 0.81, recall 1.00, f1 0.90) and a strange "micro avg" row. → **Conclusion:** the report masks the problem! The parameter `labels=np.unique(predictions)` restricts the report to the classes the model predicted — and it predicted only 5★. The full report (without `labels`) would show precision 0 and recall 0 for 1★ (with an *UndefinedMetricWarning*). Rule: confusion matrix first.

WHY DID TF-IDF "BREAK" THE MODEL? (EXTRA TEST, SAME DATA)

The notebook only states that TF-IDF made the result worse. The cause lies in combining TF-IDF with MultinomialNB and the smoothing `alpha=1`:

- TF-IDF weights are small fractions: per review they add up to about 7 in total, instead of about 113 word counts.
- The total TF-IDF "mass" of all 1★ reviews in the training set is about 4,100, and Laplace smoothing adds 1 to each of the 16,233 words in the vocabulary — so the smoothing is about 4× larger than the data of this class.
- The word distribution of the 1★ class gets "blurred" almost to uniform. Then even neutral words speak in favour of 5★ — that happened for 99.8% of the test reviews.
- Proof: `fit_prior=False` (switching off the effect of the class proportions) does not help — 2 of 228 1★ reviews detected; less smoothing helps: `alpha=0.01` → 121 of 228. What worked best was TF-IDF + `LogisticRegression(class_weight='balanced')`: 93.4% accuracy, recall 1★ = 0.88 (201 of 228), balanced accuracy 0.91.

WHAT TO WATCH OUT FOR

- With imbalanced classes always compare accuracy with the baseline (here 81%) and look at the recall of the small class and at macro avg.
- Never restrict `classification_report` with the `labels` parameter to the predicted classes.
- Fit text processing (CountVectorizer, TF-IDF) on the training set only — a Pipeline does this automatically.

CONCLUSION

Good but imbalanced data. Model 1 (Count + NB) really works: 93% against an 81% baseline, although it misses one in three negative reviews. Model 2 (TF-IDF + NB with `alpha=1`) is a constant model — its 81% is purely the majority class.

---

8 Keras — LendingClub (neural network)

File	04-Keras-Projekt-Uebung-Loesung.ipynb
Task	predict whether a borrower repays the loan (`loan_repaid`: 1 = Fully Paid, 0 = Charged Off)
Model	Keras `Sequential`: Dense 78 → 39 → 19 → 1, Dropout 0.2, sigmoid, `binary_crossentropy`, Adam, 25 epochs
Type	binary classification, imbalanced classes (80/20)
Data	396,030 loans, 27 columns; the model was trained on a 10% sample (39,522 rows)

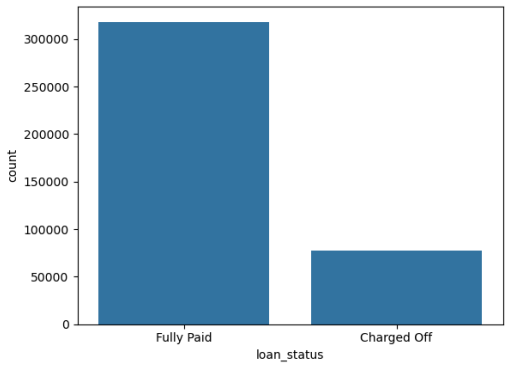
References: the file contains outputs from two runs, so some In [n] numbers repeat — go by the code snippet and the section heading (Abschnitt 1–3).

IN SHORT

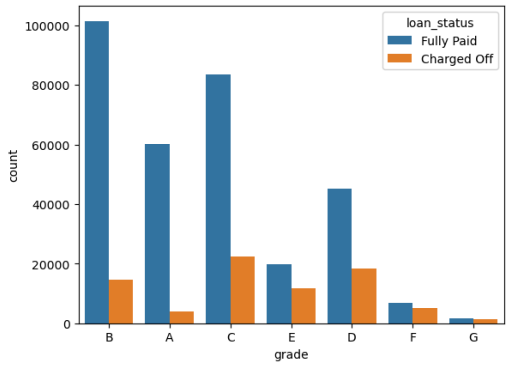
! **Data**	The raw data needed a lot of work: missing values (up to 9.5%), 15 text columns, duplicated features and one "leaking" feature. The notebook handles this in an exemplary way. Classes 80/20.
✓ **Training**	loss/val_loss curve: fast convergence; from about epoch 5 val_loss is flat, later it rises slightly → mild overfitting.
! **Test result**	Accuracy 88.1% with a baseline of 80.1% (+8 pp), but the recall for "Charged Off" is only 0.46 → the model detects fewer than half of the bad loans.

1a) Data — Abschnitt 1: Explorative Datenanalyse

`In [8]` `df.info()`	**You see:** 396,030 rows, 27 columns; Non-Null Count < 396,030 in: emp_title, emp_length, title, revol_util, mort_acc, pub_rec_bankruptcies; 15 `object` columns. → **Conclusion:** there are missing values and a lot of text — the data need preparation.
`In [9] / In [17]` `countplot(x='loan_status' ); groupby('loan_status') ['loan_amnt'].describe()`	**You see:** Fully Paid 318,357 vs Charged Off 77,673 (80.4% / 19.6% calculated). → **Conclusion:** imbalanced classes; baseline "always repays" ≈ 80%. The accuracy has to be compared with this number.
`In [16] / In [17]` `boxplot(x='loan_status', y='loan_amnt')`	**You see:** median 14,000 (Charged Off) vs 12,000 (Fully Paid); the boxes overlap strongly. → **Conclusion:** the loan amount is a weak predictor.
`In [15]` `scatterplot(x='installmen t', y='loan_amnt')`	**You see:** an almost linear relationship (the instalment follows from the amount, the interest rate and the term). → **Conclusion:** duplicated information — the notebook asks about this directly. For a network that is not an error, but the feature adds nothing new.
`In [20] and following` `countplot(x='grade', hue='loan_status'); sub_grade; F and G`	**You see:** the share of Charged Off rises steadily from A to G; in F and G it is almost 50/50. → **Conclusion:** grade/sub_grade is a strong predictor (in practice it is LendingClub's own risk rating).
`In [27]` `df.corr() ['loan_repaid'].sort_valu es().drop('loan_repaid'). plot(kind='bar')`	**You see:** the strongest is int_rate ≈ −0.25; all others |r| < 0.1. → **Conclusion:** the individual numeric features are weak — a combination of many features is needed (which justifies a neural network).
`In [11] / In [12]` `df.corr() → ValueError: could not convert string to float: ' 36 months'`	**You see:** an error instead of a heatmap. → **Conclusion:** this is an error from the newer run, not from the data: since pandas 2.0 `corr()` no longer skips text columns. Fix: `df.corr(numeric_only=True)`.



In [9] loan_status — approx. 80% Fully Paid: baseline ≈ 80%.



In [20] grade by loan_status — the share of Charged Off rises from A to G.

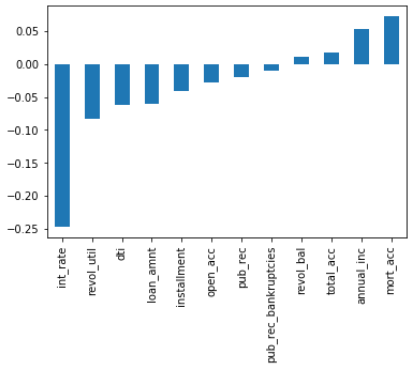
1b) Data — Abschnitt 2: Daten-Preprocessing

`In [30] / In [31]` `df.isnull().sum() / in %`	**You see:** mort_acc 9.5%, emp_title 5.8%, emp_length 4.6%, title 0.4%, pub_rec_bankruptcies 0.14%, revol_util 0.07%. → **Conclusion:** the notebook's decisions: emp_title dropped (In [33]: 173,105 unique values — impossible to encode); emp_length dropped (see below); title dropped (In [48]/[49]: it describes the same as purpose); mort_acc filled with the mean within total_acc groups (In [53]: the highest correlation, 0.38) — sensible; revol_util and pub_rec_bankruptcies: `dropna()` (< 0.5% of the rows).

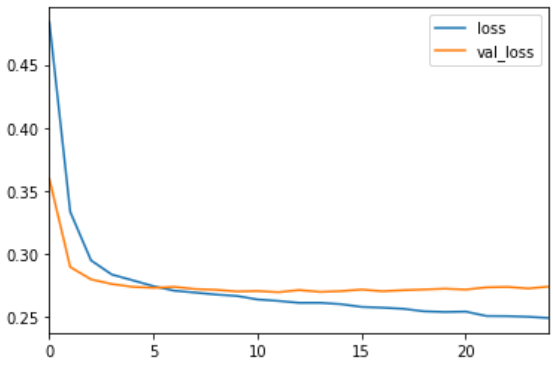
In [44] / In [45] `emp_len = emp_co / emp_fp`	**You see:** values of 0.23–0.26 in every employment-length category. → **How to read:** this is the ratio of Charged Off to Fully Paid (how many unpaid loans per repaid one), not a percentage — although the task speaks of a "Prozentanteil". The share of unpaid loans = r/(1+r) ≈ 18–21% calculated in all groups; a flat profile = a feature without information, rightly dropped.
In [64] → In [77] encoding the categories	**You see:** term → 36/60; grade dropped (contained in sub_grade); dummies: sub_grade, verification_status, application_type, initial_list_status, purpose, home_ownership (NONE/ANY → OTHER); zip_code from address → dummies; earliest_cr_line → year. → **Conclusion:** at the end only loan_status is still text (In [77]), later dropped as a duplicate of loan_repaid. The model gets 78 features — matching `Dense(78)`.
In [75] `df.drop('issue_d', axis=1)`	**You see:** the justification: "Datenüberlauf" (data leakage). → **Conclusion:** exemplary thinking: when deciding on a loan we do not know its payout date — the feature would only be available "after the fact".
In [28] `df.head() – column address`	**You see:** "Mendozaberg, OK 22690", "Loganmouth, SD 05113" and "New Sabrina, WV 05113" — the same postcode in two states. → **Conclusion:** the addresses are synthetic. Any predictive power of zip_code is an artefact of this course dataset, not a real-world pattern. Check: `df.groupby('zip_code')['loan_repaid'].mean()`.
In [82] `print(len(df)) → 39522`	**You see:** the line `df.sample(frac=0.1, …)` is now commented out, but the result of 39,522 shows that a 10% sample was used in the run the results come from. → **Conclusion:** 395,219 × 0.1 ≈ 39,522; a 20% test set = 7,905 — matching the support in the report calculated.
In [88] / In [89] `scaler.fit_transform(X_train); scaler.transform(X_test)`	**You see:** `MinMaxScaler` fitted on the training set only. → **Conclusion:** correct, no leakage (the notebook explicitly justifies it).

1c) Training — how it went

In [24] `Dense(78) … Dense(1, activation='sigmoid'); compile(loss='binary_crossentropy')`	**You see:** a funnel architecture with Dropout 0.2 after each hidden layer. → **Conclusion:** standard for binary classification; Dropout limits overfitting.
In [25] `model.fit(…, epochs=25, batch_size=256, validation_data=(X_test, y_test))`	**You see:** the test set also serves as the validation set. → **Conclusion:** when you watch the curves (and even more so with early stopping), the test set stops being independent. Correctly: a separate validation set (e.g. `validation_split=0.1`), and the test set only once, at the end.
In [97] `losses[['loss','val_loss']].plot()`	**You see:** loss falls steadily from about 0.49 to about 0.25; val_loss falls quickly to about 0.27 (about 5 epochs), then stays flat and from about epoch 15 rises slightly (to about 0.274), although loss keeps falling. → **How to read:** a growing gap between the curves = the start of overfitting (the network learns details of the training set that do not generalise). At the beginning val_loss < loss — that is normal: Dropout works only during training, and loss is the average over the whole epoch. About 5–10 epochs would have been enough; `EarlyStopping(monitor='val_loss', patience=3)` would have stopped the training automatically. The overfitting is mild (a gap of about 0.025), so the results remain reliable.



In [27] correlation of the features with loan_repaid — only int_rate clearly (≈ −0.25).



In [97] loss (training) vs val_loss — val_loss flat from about epoch 5, the gap grows.

2) Results on the test data — how to read them

In [101] — confusion_matrix(y_test, predictions)

	pred. 0	pred. 1
actual: 0 Charged Off	723	847
actual: 1 Fully Paid	95	6,240

847 = bad loans classified as good (the most expensive error for the lender).

Accuracy
88.1%
6,963 of 7,905

Baseline
80.1%
calculated

Recall Charged Off
0.46
847 missed

Balanced acc.
0.72
calculated

`In [101]`

`class encoding`

You see: class 0 = Charged Off (loan not repaid), 1 = Fully Paid; rows = truth, columns = prediction. → **Conclusion:** careful: here "class 0" is the more important one (risk).

`In [100]`

`classification_report(y_test, predictions)`

You see: class 0: precision 0.88, recall 0.46; class 1: precision 0.88, recall 0.99; accuracy 0.88; macro avg recall 0.72 / F1 0.77; weighted 0.88 / 0.87. → **How to read:** the model detects 46% of the bad loans (723 of 1,570); when it warns "will not repay", it is right in 88% of cases. It recognises almost all good loans (99%). The difference between macro and weighted = weakness on the smaller class.

baseline

You see: 6,335 / 7,905 = 80.1% calculated. → **Conclusion:** the model's 88.1% = +8 pp. A real but moderate improvement.

business interpretation

You see: calculated: if the bank granted loans only to those to whom the model gives "1": of 7,087 accepted, 847 would not repay — 12.0% instead of 19.9% without the model. 818 applications would be rejected, of which 95 (11.6%) are good customers. → **Conclusion:** this is the kind of language worth describing the result in: by how much the share of bad loans falls and how many good customers we lose.

`In [99]`

`model.predict_classes(X_test)`

You see: the default threshold of 0.5. → **Conclusion:** with imbalanced classes it can be shifted (e.g. "will repay" only at p > 0.6–0.7): more bad loans detected at the cost of rejecting more good ones. The choice depends on the costs of the two errors — the precision-recall curve helps.

`In [102] → In [104]`

`new_customer … predict … loan_repaid`

You see: the model predicts 1, the truth is 1. → **Conclusion:** one example proves nothing — an "always 1" model would also be right here (and it is right in 80% of cases). Besides, this customer comes from df, so with a probability of about 80% they were in the training set — this is not a "new" customer.

TECHNICAL STATE OF THE FILE (IMPORTANT WHEN RE-RUNNING IT)

- The newer run (14.10.2026, Python 3.13.9) covers In [1]–[21] (EDA) and In [22]–[29] (the model cells at the end); the other outputs — including the loss plot, the report and the matrix — come from an older run.
- In [11]/[12] `df.corr()` → ValueError: in pandas ≥ 2.0 use `df.corr(numeric_only=True)`.
- In [25] `fit` → NameError (X_train does not exist); In [26] → NameError (train_test_split is imported only in In [27]); In [29] → KeyError 'loan_repaid' (the whole data preparation was skipped). Solution: *Kernel → Restart & Run All*, i.e. everything in order.
- `model.predict_classes` does not exist in the new TensorFlow (removed in 2.6): `(model.predict(X_test) > 0.5).astype(int)`.
- `df.groupby('total_acc').mean()['mort_acc']` → in pandas ≥ 2.0: `df.groupby('total_acc')['mort_acc'].mean()`.

CONCLUSION

The data needed a lot of work (missing values, categories, leakage) — the notebook does it in an exemplary way. The training went correctly, with mild overfitting from about epoch 10–15. Result: +8 pp above the baseline, excellent recognition of good loans, but only 46% of the bad ones detected. Before practical use, the threshold or the class weights must be tuned.

9 Comparison of all models and the key lessons

One overview of all results on the test data. The most important thing is the comparison with the **baseline** — the result of a "model" that always chooses the most frequent class.

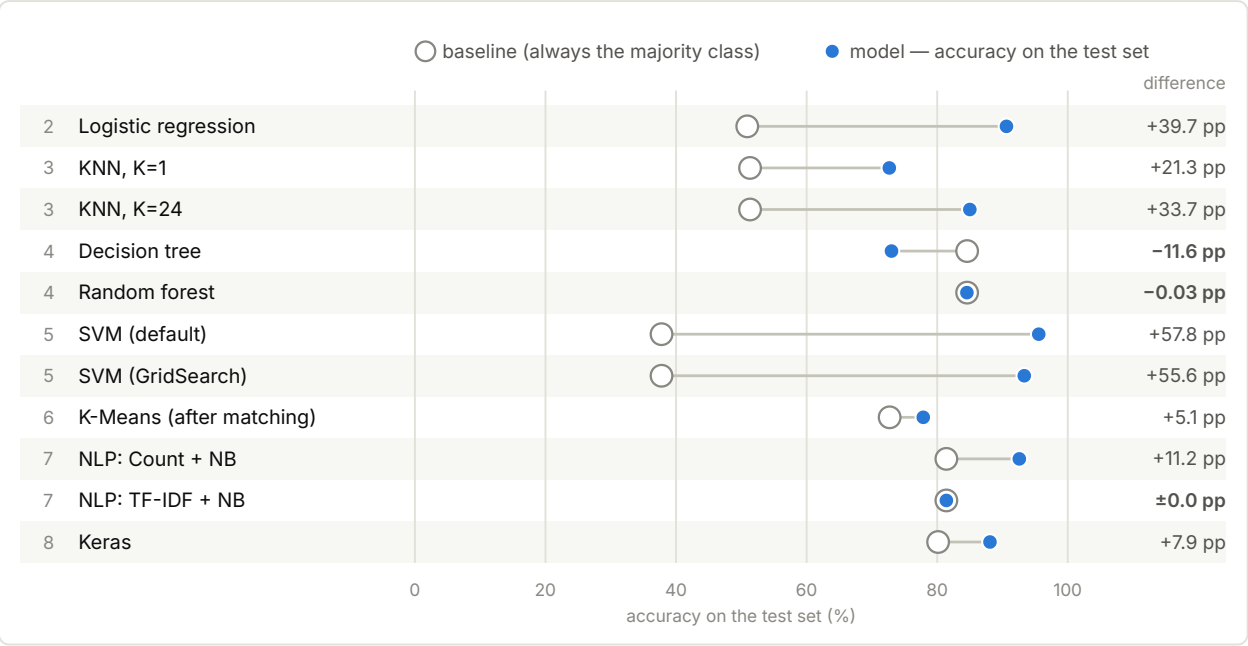


Chart: accuracy on the test set (filled dot) vs baseline (empty circle); on the right the difference in percentage points. K-Means — after matching the cluster names (calculated). Linear regression has no accuracy, so it appears only in the table.

	NOTEBOOK	DATA	TEST RESULT	BASELINE	BALANCED ACC.	VERDICT
1	**Linear regression** regression / LinearRegression	✓ clean, synthetic	RMSE $8.93 · R^2 ≈ 0.99	RMSE ≈ $79	–	✓ very good
2	**Logistic regression** classification / LogisticRegression	✓ balanced, separable	90.6%	50.9%	0.91	✓ very good

NOTEBOOK	DATA	TEST RESULT	BASELINE	BALANCED ACC.	VERDICT
3 **KNN** classification / KNN, K=24	! classes overlap	85.0% (K=1: 72.7%)	51.3%	0.85	✓ good, slightly optimistic
4 **Decision tree** classification	! 16% class 1, weak signal	73.0% · recall(1) 0.24	84.6%	0.53	✗ below baseline
4 **Random forest** classification	! as above	84.55% · recall(1) 0.02	84.59%	0.51	✗ = baseline
5 **SVM** 3-class classification / SVC	✓ exemplary	95.6% / 93.3%	37.8%	0.96 / 0.94	✓ very good
6 **K-Means** clustering	! errors, different scales	77.9% after matching (notebook: 22.1%)	72.7%	0.64	✗ weak
7 **NLP model 1** text / Count + NB	! imbalanced	92.6% · recall(1★) 0.70	81.4%	0.84	✓ good
7 **NLP model 2** text / TF-IDF + NB	! as above	81.4% · recall(1★) 0	81.4%	0.50	✗ constant model
8 **Keras** neural network	! missing values, 80/20	88.1% · recall(Charged Off) 0.46	80.1%	0.72	! moderate

The key lessons from the 8 notebooks

1. **Before you celebrate the accuracy, compute the baseline.** The random forest (84.55%) and NLP with TF-IDF (81.4%) score exactly as much as a "model" that always chooses the majority class (ch. 4, 7).
2. **Confusion matrix first, then the report.** A column of zeros only = a class the model never predicts (ch. 7). The report can be "prettified" with the `labels` parameter — the matrix cannot.
3. **With imbalanced classes, the small class is what counts:** its recall and precision, and macro avg / balanced accuracy (ch. 4, 7, 8).
4. **`describe()` → min and max of every column.** Impossible values (Grad.Rate 118, PhD 103) are visible immediately (ch. 6).
5. **Distance-based methods (KNN, K-Means, SVM, networks) need scaling** — and the scaler, vectorizer and imputation are fitted on the training data only (ch. 3, 7 vs 8).
6. **The test set is a one-time exam:** do not choose K, hyperparameters or the number of epochs on it (ch. 3, 8).
7. **A small test set = large uncertainty.** A difference of one example out of 45 is noise (ch. 5).
8. **In clustering, the cluster numbers mean nothing** — compare with labels via the best matching or ARI (ch. 6).
9. **The EDA predicts the result:** separated clouds on the pairplot → high accuracy (ch. 2, 5); overlapping distributions → weak model (ch. 4).
10. **One correct prediction proves nothing** (the "new customer" in ch. 8).

10 Technical notes for re-running the notebooks

Some of the results in the notebooks were produced with older library versions, while the metadata of several files already record a Python 3.13 environment. In new versions the following places give an error or a different result.

PROBLEM	WHERE	FIX
`df.corr()` or `groupby(…).mean()` on data with text columns → ValueError / TypeError (pandas ≥ 2.0)	Keras In [11], [12], [27], [53], [54]; NLP In [11]	`df.corr(numeric_only=True)`, `groupby(…).mean(numeric_only=True)` or selecting the column: `groupby('total_acc')['mort_acc'].mean()`
Chained assignment `df['col']['row'] = …` changes nothing (pandas ≥ 3.0, Copy-on-Write)	K-Means In [33]	`df.loc['Cazenovia College', 'Grad.Rate'] = 100`
Positional arguments in seaborn (`lmplot('a', 'b', …)`, `kdeplot(x, y)`, `shade=`) → error in seaborn ≥ 0.12	K-Means In [28], [29]; SVM In [7]	`sns.lmplot(x='Room.Board', y='Grad.Rate', data=df, …)`; `sns.kdeplot(x=…, y=…, fill=True, thresh=0.05)`
`sns.distplot` is deprecated	Keras In [10]	`sns.histplot(df['loan_amnt'], bins=40)`
`model.predict_classes` removed (TensorFlow ≥ 2.6)	Keras In [99], [103]	`(model.predict(X) > 0.5).astype(int)`
A different `classification_report` layout ("avg / total")	KNN In [30], [33]	only the format of an older version; today: accuracy / macro avg / weighted avg
No `random_state` → different numbers after every run	KNN In [23]; SVM In [9]; K-Means In [37]; trees In [18], [25]	add `random_state=…` in the split and in the model if you want to compare results
Cells executed out of order → NameError / KeyError	Keras In [25], [26], [29]	*Kernel → Restart & Run All*